Author: Hari Raja Mohan
Student ID: 24067150
Subject: Machine Learning and Neural Networks
Module Leader: Peter Scicluna
GitHub Link: https://github.com/HARIRAJAMOHAN2000/Decision-Tree-Modelling-Hyperparameter-Tuning-and-Pruning-Using-Permit-Score-Data.git

A Professional Tutorial Report on Decision Tree Modelling, Hyperparameter Tuning, and Pruning Using Permit Score Data

1. Introduction – Comprehensive Decision Tree Workflow for the Permit Score Dataset

One of the most understandable and straightforward supervised learning methods is the decision tree. They are attractive for areas that need transparency, like public policy, auditing procedures, and compliance analysis, because of their rule-based structure. Their ease of use, however, has a drawback: if unchecked, they become very deep, which results in poor generalization and memorization of training data.

This tutorial presents a complete, reproducible workflow for analysing the Permit Score dataset using a Decision Tree classifier. It demonstrates:

- construction of a preprocessing-and-model pipeline
- evaluation of a baseline model
- hyperparameter tuning using validation curves
- diagnosing model behaviour using learning curves
- cost-complexity pruning
- interpretability via feature importance

In addition to developing a precise model, the goal is to comprehend how decision tree complexity affects performance via the bias-variance trade-off.

2. Dataset Overview and Preprocessing

The dataset ontains the target variable Score and two categorical features:

- **AHJ_Name**
- **AHJ_State**

Both predictors are non-numeric and need to be changed before modeling, according to an exploratory overview (such as df.shape, df.info). The use of criteria beyond simple correctness is supported by a preliminary class distribution check that reveals a discernible imbalance.



Fig 2.1: Distribution of Permit Scores

An overview of the target variable Score's frequency distribution is shown in this graphic. With Scores 1, 2, and 3 appearing far more frequently than Scores 0 and 4, the plot illustrates the disparity among the five score categories. The performance of classification models and the interpretation of metrics like accuracy and macro F1-score are directly impacted by class imbalance, so understanding this distribution is crucial.

2.1 Preprocessing Strategy

A ColumnTransformer is used to guarantee clear and leak-free modeling:

- AHJ_Name and AHJ_State via OneHotEncoder
- The target's pass-through behaviour

This transformer's integration into a pipeline ensures that:

- Preprocessing is the same for testing and cross-validation.
- Transformations never "peek" at data that isn't visible.
- The outcomes are still modular and repeatable.

3. Decision Tree Complexity and the Bias–Variance Trade-off

Decision Trees produce hierarchical rules by recursively splitting features. The dominant hyperparameter controlling this growth is max_depth.

- **Shallow trees** (e.g., depth ≤ 2) exhibit high bias and underfit.
- **Deep trees** (large depth or unrestricted growth) exhibit high variance, fitting noise instead of patterns.

The relationship between bias and variation can be summed up as follows:

Depth	**Bias**	Variance	Typical Outcome
Low	High	Low	Underfitting
High	Low	High	Overfitting

In order to evaluate validation and learning curves in subsequent sections, it is essential to comprehend this dynamic.

4. Mathematical Foundations of Decision Trees

4.1 Node Impurity

Decision trees use an impurity metric to gauge a node's "mixedness."

- **Entropy:**

$$H(p) = -\sum p_i \log_2(p_i)$$

- **Gini Impurity:**

$$G(p) = 1 - \sum p_i^2$$

Scikit-learn uses Gini by default since it is less computationally demanding. The assumption that nodes with mixed classes have higher impurity is reflected in both.

4.2 Information Gain

The improvement in impurity is used to assess a candidate split:

$$IG = I(\text{parent}) - \sum_k \frac{n_k}{n} I(\text{child}_k)$$

The algorithm gradually increases node purity by selecting the split with the biggest gain.

5. Establishing a Baseline Model

A DummyClassifier that predicts the most common class is trained prior to creating a complete classifier. This baseline aids in determining if the Decision Tree only represents class frequency or actually learns significant structure.

To ensure a fair comparison, the Decision Tree and the dummy model are both integrated into the same preprocessing pipeline.

6. Validation Curve

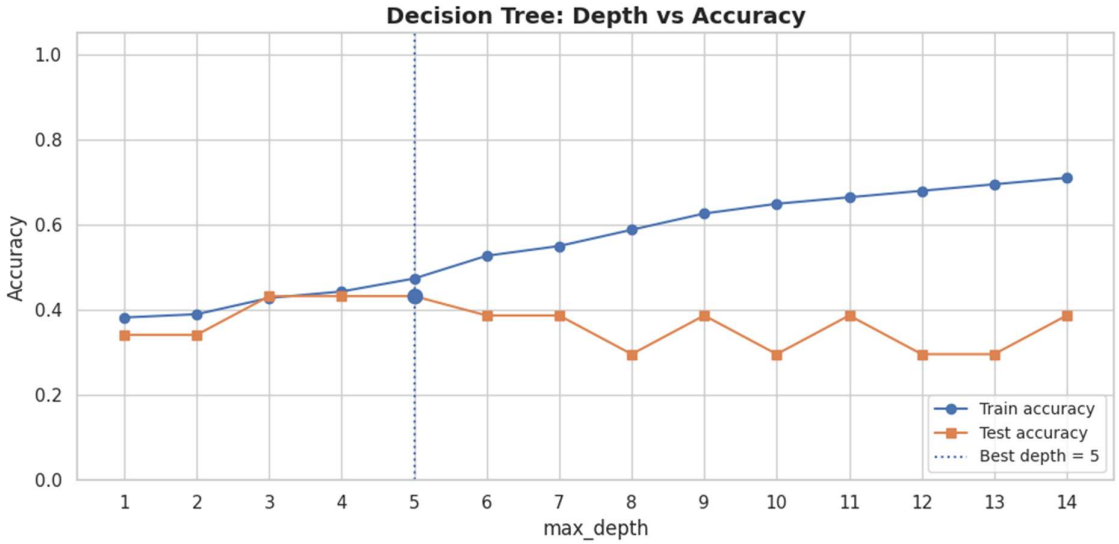


Fig 6.1: Validation Curve – Accuracy vs Tree Depth

The validation curve illustrates how the Decision Tree's performance changes as the model's maximum depth is varied, providing direct insight into the relationship between model complexity and generalisation. By plotting both training accuracy and validation accuracy across a range of depth values, the curve helps diagnose whether the model is underfitting, optimally fitted, or overfitting.

Typical Observations

- As depth increases, training accuracy rises monotonically (deep trees fit everything).
- Validation accuracy rises at first, then falls after reaching an ideal depth (usually between 5 and 7). This pattern illustrates the change from underfitting to optimal complexity to overfitting.

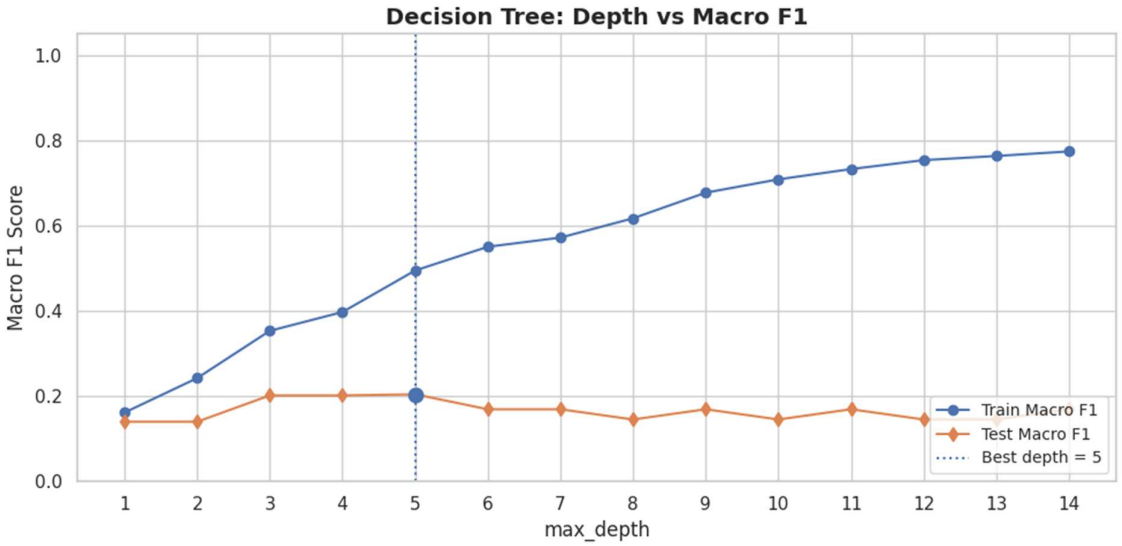


Fig 6.2: Validation Curve – Depth vs Macro F1

The Macro F1 curve illustrates how depth affects performance that is class-balanced. The test F1-score peaks at a reasonable depth and then declines, whereas training F1 increases as the tree gets deeper. The notion that overly deep trees overfit the data and perform worse on underrepresented classes is supported by this.

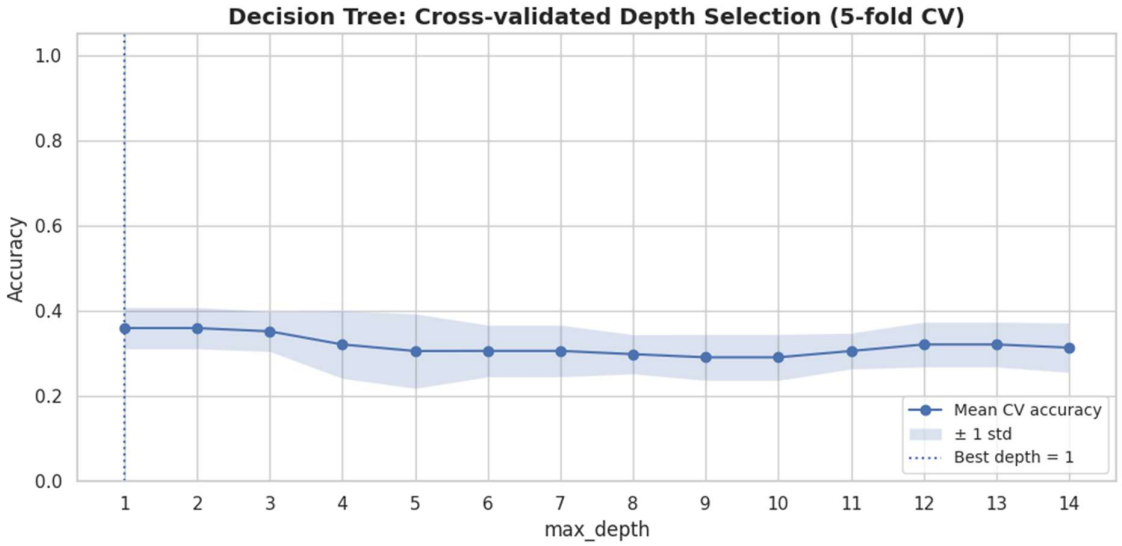


Fig 6.3: Cross Validated Depth Selection (5 fold CV)

The mean 5-fold cross-validated accuracy for various max_depth settings is shown in this plot. The ideal level of complexity for the model under cross-validation is indicated by the shallow depth that yields the highest validation accuracy. Variability across folds is captured by the shaded region, which represents ±1 standard deviation.

7. Learning Curves: Diagnosing Bias and Variance

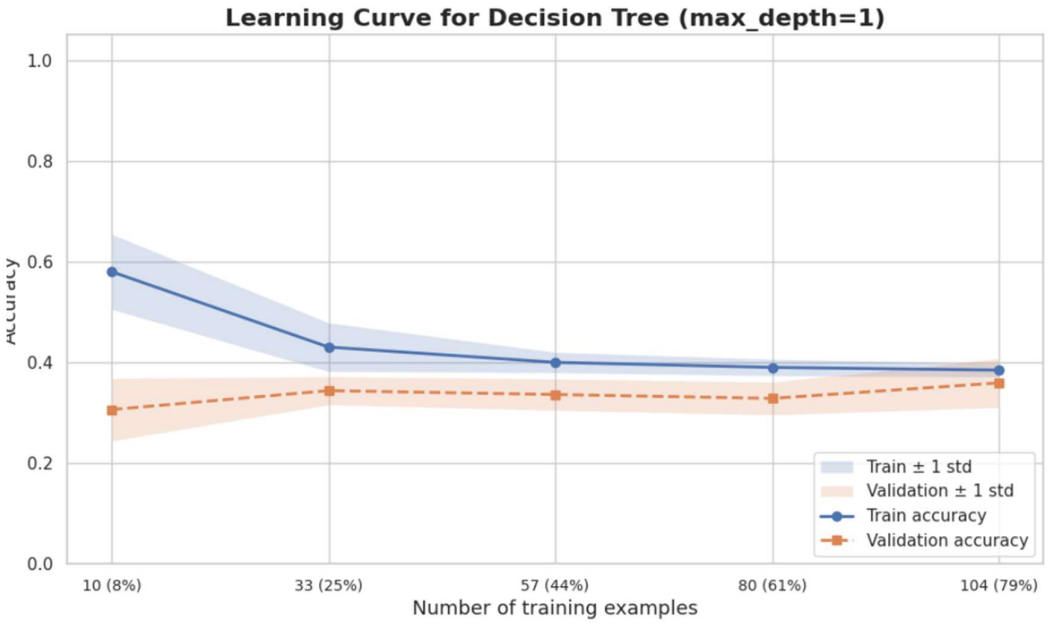


Fig 7: Bias and Variance

This learning curve shows how model performance evolves as the training set size increases. The gap between training and validation accuracy remains moderate across all sample sizes, indicating relatively low variance. However, both curves converge at a modest accuracy level, signalling the presence of high bias due to the shallow tree structure. The slight improvement in validation accuracy with more data suggests that increasing the dataset may offer limited benefits unless model capacity is increased. This supports earlier findings that the pruned tree trades complexity for interpretability at the cost of higher bias.

Model performance is illustrated via learning curves as the training sample size grows.

Interpretation Guide

- The model has a strong bias if the training and validation accuracy converge at a low value.
- The model has a significant variance if there is a continuous gap between the curves.
- A well-regularized model is indicated by curves that converge with high precision.

The learning curves frequently display stabilization for the previously determined tuned depth, indicating that more data may only slightly improve the situation.

8. Hyperparameter Tuning Using GridSearchCV

Beyond depth, a number of hyperparameters have a big impact on how trees behave:

Hyperparameter	Effect
min_samples_split	**Prevents excessive splitting on small subsets**
min_samples_leaf	**Smooths leaf behaviour; acts as regularisation**
criterion	**Chooses Gini or Entropy impurity measure**

To prevent data leaking, the entire preprocessing and model pipeline is adjusted during the cross-validation cycle using GridSearchCV. Regularising parameters (e.g., increasing min_samples_leaf) help counter overfitting while also simplifying the tree structure.

9. Mathematical Perspective on Overfitting and Capacity

9.1 Splits in Categorical Features

The number of binary splits that can occur when a categorical feature has k categories is:

$$2^{k-1} - 1$$

Large domain sizes, like numerous distinct AHJ names, significantly increase the hypothesis space and raise the danger of overfitting.

9.2 Depth and Hypothesis Space

A binary tree with depth d may include up to:

$$2^d \text{ leaves}$$

The ease with which deep trees fit noise can be explained by this exponential expansion.

10. Cost-Complexity Pruning

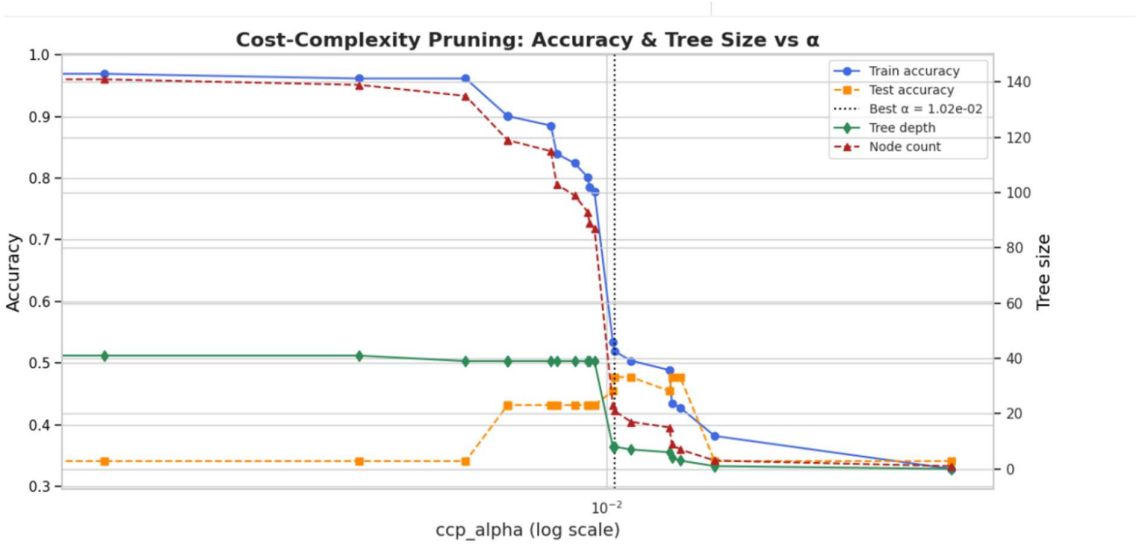


Fig 10: Cost Complexity Purning

The accuracy and structural complexity of the Decision Tree as the pruning parameter α grows are shown in this figure. The tree stays large with excellent training accuracy but decreased test accuracy for very small α values, indicating overfitting. Both depth and node count drastically decline as α rises, producing simpler trees with superior generalization. The best trade-off between accuracy and model complexity is represented by the optimal α value, which is shown by the vertical dashed line. Excessive pruning after this stage results in a decrease in test and train accuracy, which indicates underfitting.

The goal of pruning is to make the tree simpler and more adaptable.

In CART pruning, the algorithm minimises:

$$R_\alpha(T) = R(T) + \alpha|T|$$

Where:

$R(T)$ is misclassification cost

$|T|$ is number of leaves

α penalises tree size

Methodology

1. Grow an unpruned, full tree.
2. Determine the values of the pruning path for α.
3. Train subtrees for every αα.
4. To choose the subtree with the best validation performance, use cross-validation.

Small α → huge tree

large α → oversimplified model;

moderate α → optimal performance.

11. Model Interpretability

Even after being adjusted and trimmed, decision trees maintain their interpretability.

11.1 Feature Importance

One-hot encoding increases the two features in the raw dataset into several binary variables. The regional or jurisdictional identifiers that have a significant impact on permit scoring are highlighted by feature importance.

11.2 Tree Visualization

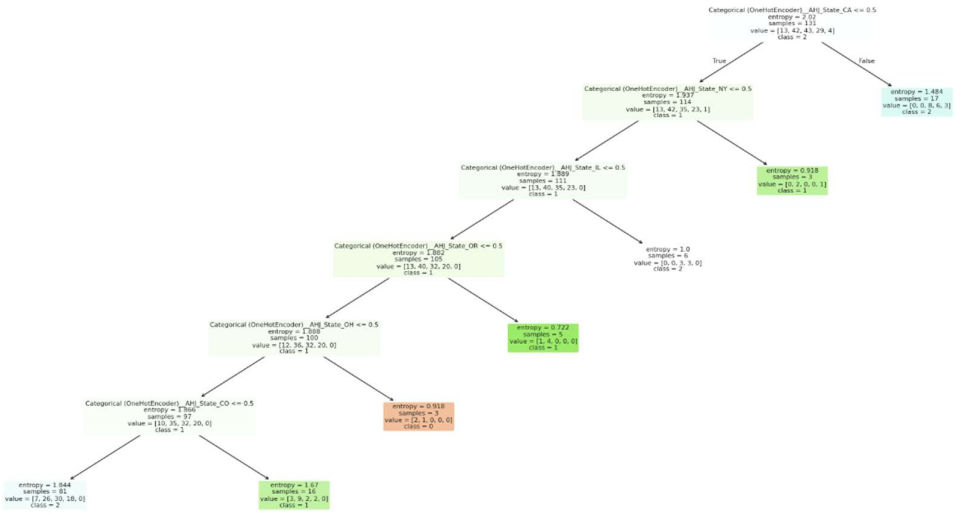


Fig 11.2 Decision Tree

An intuitive understanding of how the model makes predictions can be obtained by visualizing the trained Decision Tree's structure. Each decision node, the splitting feature, the threshold condition, and the class distribution within the resultant nodes are all shown in the tree diagram. Many of the splits match binary encoded jurisdictional traits (e.g., AHJ_State_CA, AHJ_State_NY) since the dataset includes category variables that were one-hot encoded.

12. Impact of Pruning on Model Performance

After computing the pruning path and evaluating αvalues:

- extremely small α→ overfitted, unnecessarily large models
- extremely large α→ shallow, underfitted models
- intermediate α→ highest validation accuracy

Comparing pre-pruned vs post-pruned trees typically shows:

- reduced number of leaves
- improved generalisation
- simplified visual structure
- This illustrates the usefulness of trimming in actual machine learning workflows.

13. Conclusion

Using the Permit Score dataset, this tutorial provided a thorough investigation of Decision Tree modelling. The course demonstrated how model complexity directly affects generalization behaviour through validation curves, learning curves, hyperparameter adjustment, and cost-complexity pruning.

Important lessons include:

- To prevent overfitting, tree depth needs to be managed.
- Validation curves shed light on the ideal point of complexity.
- Learning curves aid in the diagnosis of variation and bias.
- Cost-complexity pruning frequently increases interpretability while improving performance.
- Pipelines guard against data leakage and guarantee reproducibility.

The techniques presented provide a solid basis for expanding analysis to ensemble models that reduce variance while maintaining interpretability through aggregated feature importance, like Random Forests or Gradient Boosting Machines.

14. Appendix

14.1 Hyperparameters Used

- max_depth
- min_samples_split
- min_samples_leaf
- criterion
- ccp_alpha

14.2 Evaluation Metrics

- **Accuracy**
- **Macro F1-score** (important under class imbalance)
- **Confusion Matrix**
- **Cross-Validation Accuracy**

Confusion Matrix:

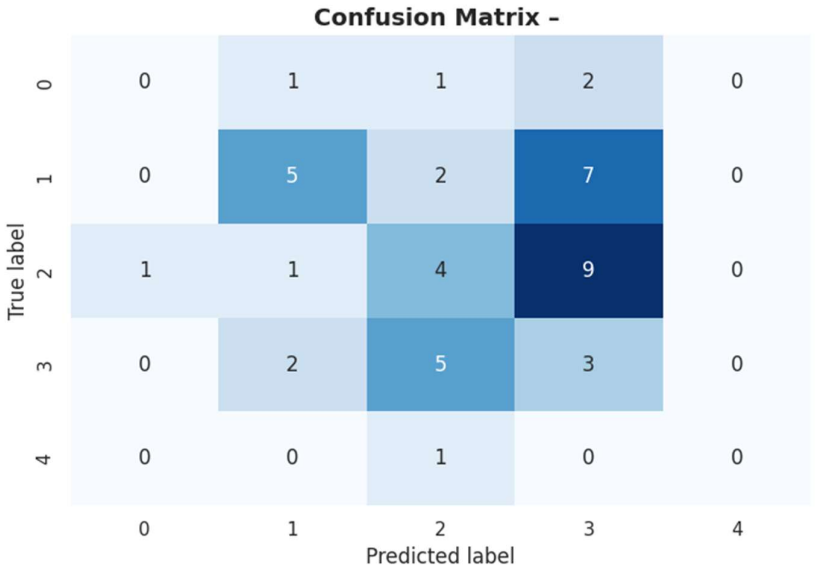


Fig 14.2.1 Confusion Matrix

By displaying the distribution of true vs predicted class labels, the confusion matrix offers a thorough assessment of the classifier's performance. The confusion matrix shows how well the model differentiates between each of the five Permit Score categories, in contrast to overall accuracy, which summarizes performance with a single score.

Both accurate classifications along the diagonal and incorrect classifications in the off-diagonal cells are highlighted in the matrix. This makes it easier to see which classes the model finds more challenging to distinguish. For instance, overlapping feature characteristics or class imbalance within the dataset may cause neighboring score categories to show greater confusion.

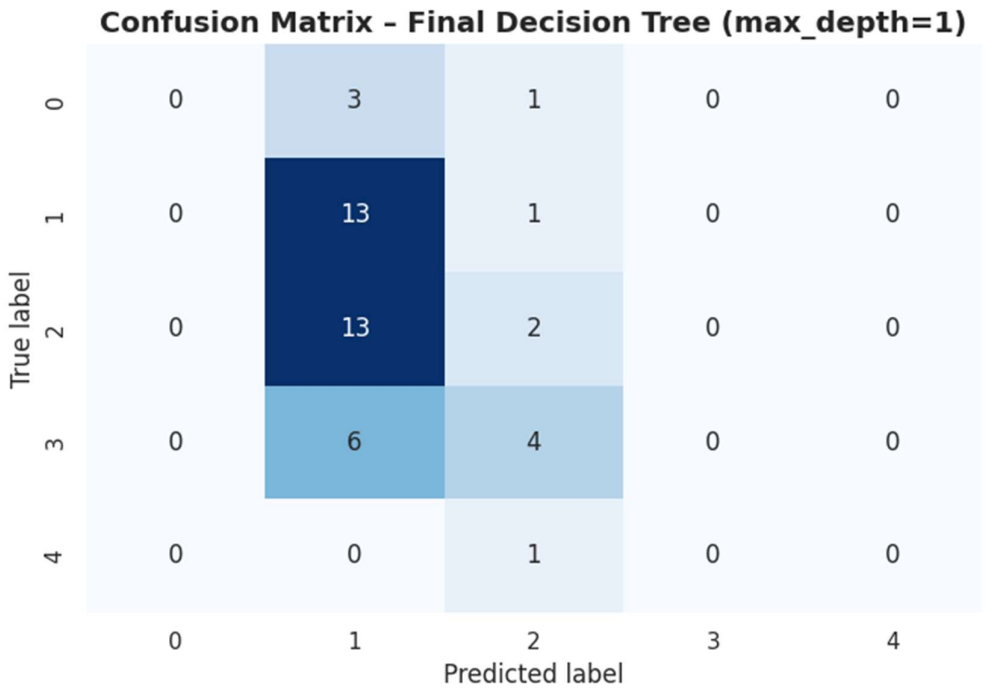


Fig 14.2.2 : Confusion Matrix (Final Decision Tree) max_depth = 1

14.3 Steps to Reproduce

1. Load the dataset and dependencies.
2. Create preprocessing and baseline models.
3. Create validation curves for tree depth.
4. Analyze learning curves.
5. Use GridSearchCV to tune hyperparameters.
6. Calculate pruning paths and evaluate α values.
7. Visualize the ultimate tree and the importance of each attribute.

14.4 Interpretability Notes

• Domain experts can better comprehend jurisdictional aspects influencing scores by using feature importance.

• Pruned models facilitate transparent decision-making by striking a compromise between clarity and accuracy.

15. References

1. **Scikit-Learn: Machine Learning in Python**
   Pedregosa, F. et al. (2011). *Scikit-Learn: Machine Learning in Python*, Journal of Machine Learning Research.
   https://jmlr.org/papers/volume12/pedregosa11a/pedregosa11a.pdf

2. **Decision Trees**
   Quinlan, J.R. (1986). *Induction of Decision Trees*. Machine Learning, 1(1):81–106.
   https://link.springer.com/article/10.1007/BF00116251

3. **Classification and Regression Trees (CART)**
   Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). *Classification and Regression Trees.*, Wadsworth.
   https://www.amazon.com/Classification-Regression-Wadsworth-Statistics-Information/dp/0412048418

4. **Bias-Variance Tradeoff**
   Geman, S., Bienenstock, E., & Doursat, R. (1992). *Neural Networks and the Bias/Variance Dilemma*. Neural Computation, 4(1):1–58.
   https://direct.mit.edu/neco/article/4/1/1/5810/

5. **One-Hot Encoding Explanation**
   Bishop, C.M. (2006). *Pattern Recognition and Machine Learning*. Springer.
   https://www.springer.com/gp/book/9780387310732

6. **Cross-Validation and Model Selection**
   Arlot, S. & Celisse, A. (2010). *A Survey of Cross-Validation Procedures for Model Selection*. Statistics Surveys, 4:40–79.
   https://projecteuclid.org/journals/statistics-surveys/volume-4/issue-none/A-Survey-of-Cross-Validation-Procedures-for-Model-Selection/10.1214/09-SS054.full